

Documentación Técnica - Proyecto Unity

Overwatch SAS

29 de marzo de 2026

Índice

1 SATCS — Overview del Proyecto	5
1.1 ¿Qué es SATCS?	5
1.2 ¿Qué hace el jugador?	5
1.3 Plataforma y tecnología	5
1.4 Escenarios disponibles	6
1.5 Las 5 Etapas	6
1.6 ¿Para quién es esta documentación?	6
2 SATCS — Arquitectura del Sistema	8
2.1 Visión general	8
2.2 Patrones de diseño usados	9
2.2.1 Singleton	9
2.2.2 Eventos C# (Action)	9
2.2.3 ScriptableObjects	9
2.2.4 jslib (JavaScript ↔ Unity)	9
2.3 Módulos del sistema	10
2.3.1 Módulo de Sensores	10
2.3.2 Módulo de Cámara / Movimiento	10
2.3.3 Módulo de Progresión	10
2.3.4 Módulo de Hotspots	10
2.3.5 Módulo de Interacción	11
2.3.6 Módulo de Retroalimentación Ambiental	11
2.3.7 Módulo de UI / Calibración	11
2.4 Flujo de datos de un hotspot	11
3 SATCS — Documentación de Scripts	13
3.1 GPSManager	13
3.1.1 Propiedades públicas	13
3.1.2 Notas	13
3.2 GyroscopeManager	14
3.2.1 Propiedades y métodos públicos	14
3.3 ARCameraController	14
3.3.1 Inspector	14
3.3.2 Métodos públicos	15
3.3.3 CharacterController (componente adicional requerido)	15
3.4 JoystickController	15

3.4.1	Inspector	15
3.4.2	Propiedad pública	16
3.5	StageManager	16
3.5.1	Etapas (enum Stage)	16
3.5.2	Inspector	16
3.5.3	Evento público	16
3.5.4	Métodos públicos	16
3.6	HotspotData	16
3.6.1	Campos	17
3.6.2	actionType	17
3.7	HotspotController	17
3.7.1	Inspector	18
3.7.2	Método público	18
3.8	NpcDialogueData	18
3.8.1	Campos	18
3.8.2	DialogueOption	18
3.9	NpcDialoguePanel	18
3.9.1	Inspector	19
3.9.2	API pública	19
3.10	SiataCallPanel	19
3.10.1	Inspector	19
3.11	CinematicManager	19
3.11.1	Inspector	20
3.11.2	Videos en WebGL	20
3.12	CriticalModePanel	20
3.12.1	Inspector	20
3.13	AudioStageManager	20
3.13.1	Inspector	21
3.13.2	API pública	21
3.14	SensorCalibrationPanel	21
3.14.1	API pública	21
3.15	UIManager	21
3.15.1	Inspector	21
4	SATCS — Guía: Crear una Escena Nueva	23
4.1	Paso 1 — Crear la escena	23
4.2	Paso 2 — GameObjects raíz (Managers)	23
4.2.1	2.1 ARSceneSetup	23
4.2.2	2.2 StageManager	23
4.2.3	2.3 GPSManager	24
4.2.4	2.4 GyroscopeManager	24
4.2.5	2.5 CameraFeedManager	24
4.2.6	2.6 AudioStageManager	24
4.2.7	2.7 CinematicManager	24
4.3	Paso 3 — Main Camera	24
4.4	Paso 4 — Canvas Principal (AR_Canvas)	25
4.5	Paso 5 — Paneles UI dentro de AR_Canvas	25
4.5.1	5.1 JoystickPanel	25

4.5.2	5.2 HotspotPanel	25
4.5.3	5.3 WelcomePanel	26
4.5.4	5.4 NpcDialoguePanel	26
4.5.5	5.5 SiataCallPanel	26
4.5.6	5.6 SensorCalibPanel	26
4.5.7	5.7 CinematicPanel	27
4.5.8	5.8 CriticalModePanel	27
4.5.9	5.9 HUD	27
4.6	Paso 6 — Conexiones de Inspector globales	27
4.7	Paso 7 — Terreno	28
4.8	Paso 8 — Agregar hotspots	28
4.9	Paso 9 — Probar en editor	28
4.10	Paso 10 — Build WebGL	28
5	SATCS — Flujo de Trabajo	30
5.1	Agregar un Hotspot nuevo	30
5.1.1	Paso 1 — Crear el ScriptableObject HotspotData	30
5.1.2	Paso 2 — Crear el diálogo (si es NpcConversation o SiataCall)	30
5.1.3	Paso 3 — Colocar el objeto 3D en la escena	31
5.1.4	Paso 4 — Asignar al StageManager (opcional)	31
5.2	Agregar una cinemática	31
5.2.1	Paso 1 — Preparar el video	31
5.2.2	Paso 2 — Crear el HotspotData	31
5.2.3	Paso 3 — Verificar CinematicManager en escena	32
5.3	Crear un diálogo con el SIATA	32
5.4	Configurar el audio por etapa	32
5.5	Configurar qué aparece en cada etapa	32
5.6	Configurar el CriticalModePanel	33
5.7	Probar sin GPS (modo editor)	33
5.8	Forzar una etapa específica al iniciar	33
5.9	Ajustar el giroscopio (en dispositivo real)	34
5.10	Recalibrar el GPS	34
6	SATCS — Limitaciones y Consideraciones	35
6.1	Plataforma	35
6.1.1	WebGL es obligatorio	35
6.1.2	HTTPS requerido para GPS y giroscopio	35
6.1.3	iOS requiere permiso explícito de usuario	35
6.2	GPS	36
6.2.1	No funciona en el editor de Unity	36
6.2.2	Primera lectura como origen	36
6.2.3	Precisión del GPS en interiores	36
6.2.4	Escala GPS ↔ Unity	36
6.3	Video (CinematicManager)	36
6.3.1	WebGL: solo URL, no VideoClip	36
6.3.2	Editor Linux: mp4 puede fallar	36
6.3.3	Sin soporte de DRM	36
6.4	CharacterController y Terreno	37

6.4.1	El terreno debe tener Collider	37
6.4.2	Configuración del CharacterController en Main Camera	37
6.4.3	Sin soporte de salto	37
6.5	Audio	37
6.5.1	AudioStageManager necesita clips asignados	37
6.5.2	Crossfade puede sonar raro con clips muy cortos	37
6.6	Sistemas pendientes del GDD	38
6.7	Limitaciones de diseño	38
6.7.1	Un solo diálogo correcto por hotspot	38
6.7.2	El jugador no puede retroceder de etapa	38
6.7.3	Hotspots en rango al inicio	38
6.7.4	GPS y joystick no se combinan	38
6.8	Preguntas frecuentes	39
7	SATCS — Documentación Técnica	40
7.1	Índice	40
7.2	Inicio rápido	40
7.3	Contexto para IAs	41

Capítulo 1

SATCS — Overview del Proyecto

1.1 ¿Qué es SATCS?

Sistema de Alerta Temprana Comunitaria (SATCS) es una experiencia educativa interactiva en realidad aumentada que corre en el navegador web. Fue desarrollado para la **Institución Universitaria Pascual Bravo** en Medellín.

Su objetivo es enseñar a la comunidad cómo funcionan los sistemas de alerta temprana ante riesgos hídricos (quebradas, inundaciones) en barrios urbanos.

1.2 ¿Qué hace el jugador?

El jugador camina virtualmente por un barrio (usando su teléfono o computador) y:

1. Explora el entorno en primera persona
 2. Se acerca a puntos de interés (**hotspots**) en el barrio
 3. Interactúa con vecinos (NPC), llama al SIATA, ve cinemáticas explicativas
 4. Toma decisiones correctas para avanzar por 5 etapas narrativas
 5. En las etapas críticas recibe alertas visuales y sonoras
 6. Al final (Etapa 5) observa el impacto del evento desde una vista aérea
-

1.3 Plataforma y tecnología

Aspecto	Detalle
Motor	Unity 2022.3.55f1
Plataforma de salida	WebGL (navegador web)
Lenguaje	C#
Renderizado	URP / modo skybox
AR manual	GPS + giroscopio del dispositivo (sin ARFoundation)

No usa ARFoundation. La AR se construye manualmente: - El **GPS** del teléfono posiciona al jugador en el mundo 3D - El **giroscopio** rota la cámara según hacia dónde mira el usuario - En PC: el teclado (WASD) mueve al jugador y el mouse rota la cámara

1.4 Escenarios disponibles

El GDD define 3 barrios:

Escenario	Quebrada
Olaya Herrera	Quebrada La Iguaná
La Cantera Sur	Quebrada La Cantera Ramal Sur
Las Estancias	Quebrada Santa Elena

La escena actualmente activa es **CASAS_OLAYA**.

1.5 Las 5 Etapas

Cada escenario recorre 5 estados progresivos del mismo barrio:

Etapas	Nombre	Descripción
Intro	Bienvenida	Pantalla de onboarding antes de empezar
Etapas 1	Estado normal	Barrio tranquilo, primeros hotspots informativos
Etapas 2	Precipitación	Lluvia leve, cambios visuales sutiles
Etapas 3	Incremento de caudal	Alerta temprana, modo crítico activado
Etapas 4	Evento crítico	Situación de emergencia, decisiones urgentes
Etapas 5	Vista aérea	Observación del impacto desde arriba

1.6 ¿Para quién es esta documentación?

Esta documentación está dirigida a: - Desarrolladores que se incorporan al proyecto - Diseñadores que quieren agregar contenido (hotspots, diálogos, etapas) - Estudiantes o colaboradores sin experiencia avanzada en Unity

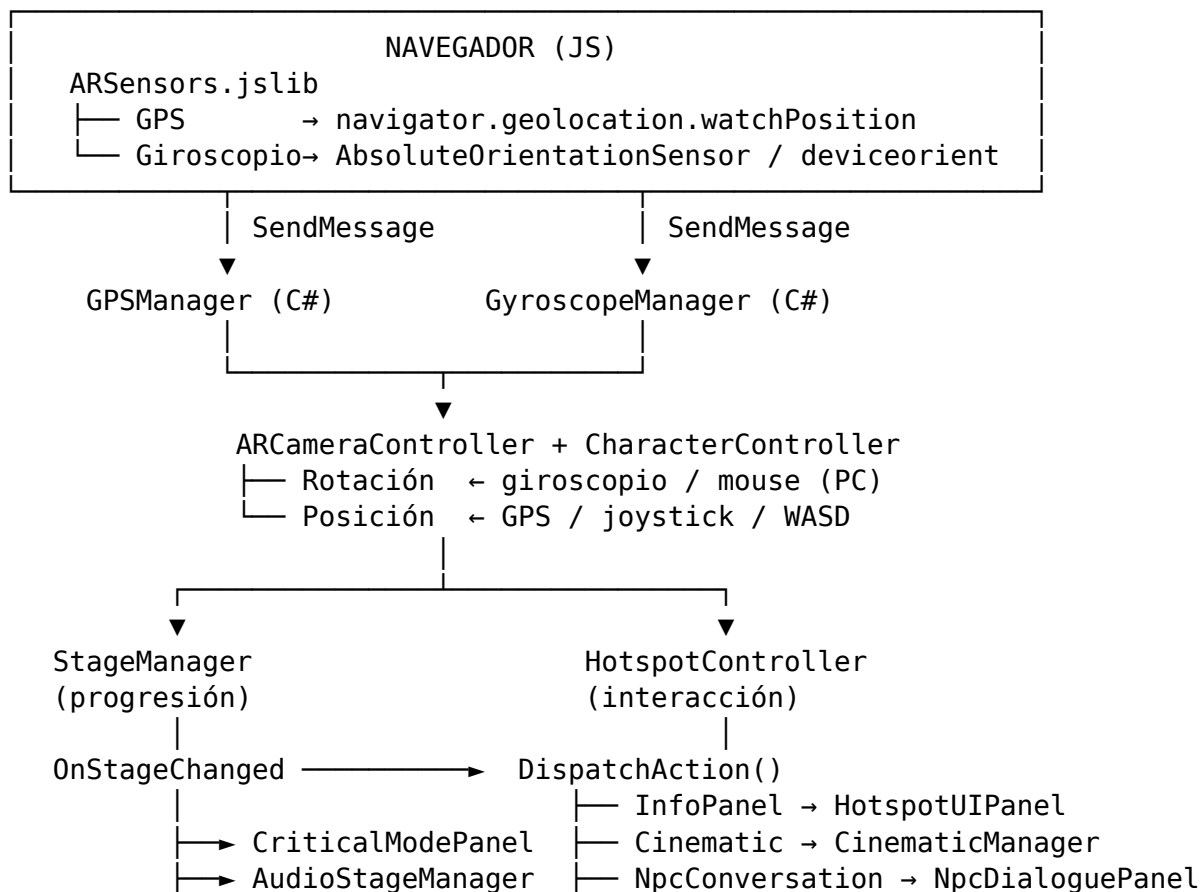
No se necesita experiencia avanzada en Unity para agregar contenido al juego.
La mayoría de los datos se configuran mediante ScriptableObjects en el Inspector, sin escribir código.

Capítulo 2

SATCS — Arquitectura del Sistema

2.1 Visión general

El proyecto está dividido en capas que se comunican entre sí mediante eventos. Ningún sistema necesita conocer los detalles internos de los otros — solo escuchan eventos o llaman a APIs públicas.



↳ UIManager

↳ SiataCall → SiataCallPanel

2.2 Patrones de diseño usados

2.2.1 Singleton

Los managers principales son Singletons con DontDestroyOnLoad. Se acceden desde cualquier script mediante NombreClase.Instance.

```
GPSManager.Instance
GyroscopeManager.Instance
StageManager.Instance
CinematicManager.Instance
CriticalModePanel.Instance
AudioStageManager.Instance
NpcDialoguePanel.Instance
SiataCallPanel.Instance
SensorCalibrationPanel.Instance
```

2.2.2 Eventos C# (Action)

Los sistemas se comunican sin acoplamiento directo. El principal es:

```
StageManager.Instance.OnStageChanged += MiMetodo;
// Firma: (Stage anterior, Stage nueva)
```

Suscriptores actuales: HotspotController, UIManager, CriticalModePanel, AudioStageManager.

2.2.3 ScriptableObjects

Los datos de contenido (hotspots, diálogos) se guardan como assets independientes. Esto permite cambiar el contenido sin tocar el código.

```
HotspotData → Assets > Create > AR > Hotspot Data
NpcDialogueData → Assets > Create > AR > NPC Dialogue Data
```

2.2.4 jslib (JavaScript ↔ Unity)

Los sensores del navegador (GPS, giroscopio) no son accesibles desde C# directamente. Se usa un plugin JavaScript (ARSensors.jslib) que envía los datos a Unity mediante SendMessage.

2.3 Módulos del sistema

2.3.1 Módulo de Sensores

Maneja la comunicación con el hardware del dispositivo.

Script	Función
ARSensors.jslib	Plugin JS que accede a GPS y giroscopio del navegador
GPSManager	Recibe GPS, calcula posición en metros (Haversine)
GyroscopeManager	Convierte ángulos del giroscopio a rotación Unity

2.3.2 Módulo de Cámara / Movimiento

Controla cómo el jugador se mueve y mira.

Script	Función
ARCameraController	Aplica GPS/joystick/WASD al movimiento; giroscopio/mouse a la rotación
JoystickController	Joystick táctil en pantalla para móviles sin GPS
CharacterController	Componente Unity que maneja colisión con el terreno y gravedad

2.3.3 Módulo de Progresión

Controla en qué etapa está el jugador y qué está activo.

Script	Función
StageManager	Enum de etapas, GoToStage(), NextStage(), OnStageChanged
WelcomePanel	Pantalla de onboarding inicial antes de Etapa1

2.3.4 Módulo de Hotspots

Los puntos de interés del mundo 3D.

Script	Función
HotspotData	ScriptableObject con el contenido del hotspot

Script	Función
HotspotController	Detecta proximidad/clic y lanza la acción correspondiente
HotspotUIPanel	Panel de texto simple (InfoPanel)

2.3.5 Módulo de Interacción

Los paneles que aparecen al activar un hotspot.

Script	Función
NpcDialoguePanel	Conversación con NPC con opciones de respuesta
SiataCallPanel	Simulación de llamada al SIATA
MultipleChoicePanel	Botones A/B/C reutilizados por los paneles de arriba
CinematicManager	Video fullscreen con botón Skip

2.3.6 Módulo de Retroalimentación Ambiental

Reacciona automáticamente a los cambios de etapa.

Script	Función
CriticalModePanel	Modal de alerta en Etapa3/4
AudioStageManager	Música/ambiente con crossfade por etapa

2.3.7 Módulo de UI / Calibración

Interfaz general del jugador.

Script	Función
UIManager	HUD con estado GPS/giroscopio, toggle joystick
SensorCalibrationPanel	Ajuste manual del giroscopio

2.4 Flujo de datos de un hotspot

Jugador se acerca al hotspot (3D)

↓

```
HotspotController.CheckProximity()
    distancia < triggerRadius → DispatchAction()
```

↓

Lee HotspotData.actionType:

```
InfoPanel      → HotspotUIPanel.Show(data)
Cinematic      → CinematicManager.Play(data, this)
NPCConversation → NPCDialoguePanel.Instance.Show(data.dialogueData, this)
SiataCall      → SiataCallPanel.Instance.Show(data.dialogueData, this)
↓
SceneManager.SetPlayerInputBlocked(true) ← el jugador no puede moverse
↓
Usuario responde correctamente
↓
SceneManager.NextStage()
SceneManager.SetPlayerInputBlocked(false)
Panel.Hide() → HotspotController.ClosePanel()
```

Capítulo 3

SATCS — Documentación de Scripts

Referencia rápida de cada script: qué hace, qué campos configurar en el Inspector y qué métodos son útiles.

3.1 GPSManager

Archivo: Assets/Scripts/Gpsmanager.cs **Tipo:** Singleton · DontDestroyOnLoad

Recibe las coordenadas GPS del navegador y las convierte en una posición 3D relativa al punto de inicio.

3.1.1 Propiedades públicas

Propiedad	Tipo	Descripción
IsAvailable	bool	GPS disponible en el dispositivo/navegador
HasOrigin	bool	Ya se recibió la primera coordenada (punto de origen)
DisplacementMeters	Vector2	Desplazamiento en metros desde el origen. X = Este, Y = Norte

3.1.2 Notas

- En el editor Unity siempre devuelve IsAvailable = false. Activar forceJoystick en ARCameraController para probar sin GPS.
- La primera coordenada recibida se guarda como origen (el jugador “nace” ahí). Las siguientes calculan el desplazamiento.

3.2 GyroscopeManager

Archivo: Assets/Scripts/Gyroscopemanager.cs **Tipo:** Singleton · DontDestroyOnLoad

Convierte los datos del giroscopio del navegador en una rotación Unity (Quaternion).

3.2.1 Propiedades y métodos públicos

Miembro	Descripción
IsAvailable	Giroscopio disponible
DeviceRotation	Quaternion suavizado listo para asignar a la cámara
Recalibrate()	Toma una nueva calibración automática en el próximo frame
SetOffset(pitch, roll, yaw)	Offset manual. Persiste en PlayerPrefs

3.3 ARCameraController

Archivo: Assets/Scripts/Arcameracontroller.cs **Tipo:** Componente en Main Camera

Controla el movimiento y la rotación del jugador. Usa CharacterController para seguir el terreno.

3.3.1 Inspector

Campo	Tipo	Descripción
joystickController	JoystickController	Asignar JoystickPanel. Se auto-detecta si queda vacío
arObject	GameObject	Objeto 3D colocado frente al jugador al iniciar
gpsToUnityScale	float	Escala metros GPS → unidades Unity (default 1)
joystickSpeed	float	Velocidad con joystick (default 5)
wasdSpeed	float	Velocidad con teclado WASD (default 5)
mouseLookSensitivity	float	Sensibilidad del mouse en PC (default 3)

Campo	Tipo	Descripción
forceJoystick	bool	Ignora GPS y usa joystick/WASD. Útil en editor
gravity	float	Gravedad aplicada (default -20)

3.3.2 Métodos públicos

Método	Descripción
SetInputBlocked(bool)	Bloquea/desbloquea movimiento y rotación
SetForceJoystick(bool)	Activa/desactiva modo joystick manualmente
Recalibrate()	Resetea GPS, giroscopio y reposiciona el arObject

3.3.3 CharacterController (componente adicional requerido)

Configurar en el mismo GameObject (Main Camera):

Campo	Valor	Por qué
Height	1.7	Altura del jugador
Center Y	-0.85	Pies a 1.7u bajo la cámara (ojos)
Slope Limit	45	Sube pendientes de hasta 45°
Step Offset	0.3	Sube escalones de 30cm

3.4 JoystickController

Archivo: Assets/Scripts/Joystickcontroller.cs **Tipo:** Componente UI
Joystick táctil en pantalla. Activo automáticamente cuando no hay GPS.

3.4.1 Inspector

Campo	Descripción
knob	RectTransform del círculo interior del joystick
knobRadius	Tamaño del área de movimiento (px)
deadZone	Zona muerta para evitar drift (0-0.3)

3.4.2 Propiedad pública

- `InputDirection` — `Vector2` normalizado. X = horizontal, Y = adelante/atrás.
-

3.5 StageManager

Archivo: `Assets/Scripts/StageManager.cs` **Tipo:** Singleton · DontDestroyOnLoad
Núcleo de la progresión. Controla en qué etapa está el jugador.

3.5.1 Etapas (enum Stage)

`Intro=0` `Etapal=1` `Etapal=2` `Etapal=3` `Etapal=4` `Etapal=5`

3.5.2 Inspector

Campo	Descripción
<code>stageConfigs[6]</code>	Una entrada por etapa. Define qué <code>GameObjects</code> activar/desactivar
<code>cameraController</code>	Main Camera. Se auto-detecta si queda vacío
<code>startStage</code>	Etapal inicial al presionar Play (normalmente <code>Intro</code>)

3.5.3 Evento público

`StageManager.Instance.OnStageChanged` += (anterior, nueva) => { ... };

3.5.4 Métodos públicos

Método	Descripción
<code>NextStage()</code>	Avanza a la siguiente etapa
<code>GoToStage(Stage)</code>	Salta directamente a una etapa
<code>SetPlayerInputBlocked(bool)</code>	Bloquea/desbloquea el movimiento del jugador

3.6 HotspotData

Archivo: `Assets/Scripts/Hotspotdata.cs` **Tipo:** `ScriptableObject`

Crear desde: **Assets > Create > AR > Hotspot Data**

Contiene todo el contenido de un punto de interés. No tiene lógica.

3.6.1 Campos

Campo	Tipo	Descripción
title	string	Título visible en el panel
description	string	Texto informativo (para InfoPanel)
icon	Sprite	Ícono opcional
actionType	enum	Qué ocurre al activarlo (ver abajo)
cinematicClip	VideoClip	Video para editor/standalone
cinematicUrl	string	URL para WebGL (ej: StreamingAssets/Videos/clip.mp4)
cinematicAdvancesStage	bool	El video avanza etapa al terminar
dialogueData	NpcDialogueData	Datos del diálogo (NPCConversation / SiataCall)
triggerRadius	float	Radio de activación en unidades Unity
allowClick	bool	También activa con tap/clic
requiredStage	int	-1 = siempre visible. 0-5 = etapa específica
isBlinking	bool	Efecto de pulso visual en el objeto 3D
blinkSpeed	float	Velocidad del pulso (Hz)

3.6.2 actionTypes

Valor	Qué ocurre
InfoPanel	Muestra panel de texto simple
Cinematic	Reproduce video fullscreen
NpcConversation	Abre diálogo con NPC y opciones
SiataCall	Simula llamada al SIATA

3.7 HotspotController

Archivo: Assets/Scripts/Hotspotcontroller.cs **Tipo:** Componente · RequireComponent(Collider)

Se adjunta a cada objeto 3D que es un punto de interés en la escena.

3.7.1 Inspector

Campo	Descripción
data	HotspotData (ScriptableObject) con el contenido
uiPanel	HotspotUIPanel de la escena. Se auto-detecta si queda vacío

3.7.2 Método público

- `ClosePanel()` — cierra el panel abierto por este hotspot (llamado por los paneles al cerrarse).

3.8 NpcDialogueData

Archivo: Assets/Scripts/NpcDialogueData.cs **Tipo:** ScriptableObject

Crear desde: **Assets > Create > AR > NPC Dialogue Data**

3.8.1 Campos

Campo	Tipo	Descripción
npcName	string	Nombre del NPC
npcPhoto	Sprite	Foto del NPC
npcText	string	Pregunta o mensaje del NPC
options[]	DialogueOption[]	Opciones de respuesta
advancesStageOnCorrect	bool	Responder bien avanza la etapa
correctAnswerDelay	float	Pausa antes de avanzar (segundos)

3.8.2 DialogueOption

Campo	Descripción
optionText	Texto del botón
isCorrect	Marcar solo en la respuesta correcta
feedbackText	Texto pedagógico mostrado tras seleccionar

3.9 NpcDialoguePanel

Archivo: Assets/Scripts/NpcDialoguePanel.cs **Tipo:** Singleton UI · inicia inactivo

3.9.1 Inspector

Campo	Apunta a
npcPhoto	Image del NPC
npcNameText	TextMeshProUGUI del nombre
dialogueText	TextMeshProUGUI del texto
choicePanel	MultipleChoicePanel (hijo del panel)

3.9.2 API pública

```
NpcDialoguePanel.Instance.Show(NpcDialogueData data, HotspotController source);  
NpcDialoguePanel.Instance.Hide();
```

3.10 SiataCallPanel

Archivo: Assets/Scripts/SiataCallPanel.cs **Tipo:** Singleton UI · inicia inactivo

Igual que NpcDialoguePanel pero con UI de llamada telefónica: encabezado de llamada, animación “Llamando...→Conectado” y botón Colgar.

3.10.1 Inspector

Campo	Apunta a
callerPhoto	Image del caller
callerNameText	TextMeshProUGUI del nombre
callStatusText	TextMeshProUGUI “Llamando...” / “Conectado”
dialogueText	TextMeshProUGUI del mensaje
choicePanel	MultipleChoicePanel (hijo del panel)
hangUpButton	Button “Colgar”
connectingDelay	Segundos de animación “Llamando...”

3.11 CinematicManager

Archivo: Assets/Scripts/CinematicManager.cs **Tipo:** Singleton · DontDestroyOnLoad

Reproductor de video fullscreen. Se activa cuando un hotspot tiene `actionType = Cinematic`.

3.11.1 Inspector

Campo	Apunta a
cinematicPanel	GameObject del panel (inactivo por defecto)
videoDisplay	RawImage donde se renderiza el video
loadingText	TextMeshProUGUI "Cargando video..."
skipButton	Button "Saltar"
videoPlayer	VideoPlayer del mismo GameObject
skipIfNoContent	Si no hay clip/URL, cierra sin error

3.11.2 Videos en WebGL

- Colocar el archivo .mp4 en Assets/StreamingAssets/Videos/
- En el HotspotData, asignar cinematicUrl = "StreamingAssets/Videos/miVideo.mp4"
- En editor Linux, usar formato .webm para preview

3.12 CriticalModePanel

Archivo: Assets/Scripts/CriticalModePanel.cs **Tipo:** Singleton UI · inicia inactivo
Modal automático que aparece al entrar a Etapa3 o Etapa4.

3.12.1 Inspector

Campo	Descripción
stageContents[]	Contenido por etapa (stage, title, description, icon)
alertIcon	Image del ícono de alerta
titleText	TextMeshProUGUI del título
descriptionText	TextMeshProUGUI de la descripción
continueButton	Button "Entendido"
showDelay	Segundos de espera antes de aparecer (default 1)

Por defecto ya viene configurado para Etapa3 y Etapa4 con textos de alerta.

3.13 AudioStageManager

Archivo: Assets/Scripts/AudioStageManager.cs **Tipo:** Singleton · DontDestroyOnLoad

Gestiona el audio ambiental con crossfade entre etapas.

3.13.1 Inspector

Campo	Descripción
stageAudios[6]	Una entrada por etapa: clip, volume, fadeTime
masterVolume	Volumen maestro global (0-1)
sourceA/B/SFX	AudioSources (se crean automáticamente si quedan vacíos)

3.13.2 API pública

```
// Sonido puntual (trueno, alerta)
AudioStageManager.Instance.PlaySFX(AudioClip clip, float volume = 1f);

// Silenciar durante cinemática
AudioStageManager.Instance.MuteAmbient(float fadeDuration = 0.5f);
AudioStageManager.Instance.RestoreVolume(float fadeDuration = 0.5f);
```

3.14 SensorCalibrationPanel

Archivo: Assets/Scripts/SensorCalibrationPanel.cs **Tipo:** Singleton UI · inicia inactivo

3 sliders para ajustar el giroscopio manualmente. Los valores persisten entre sesiones (PlayerPrefs).

3.14.1 API pública

```
SensorCalibrationPanel.Instance.Show();
```

3.15 UIManager

Archivo: Assets/Scripts/Uimanager.cs **Tipo:** Componente en HUD

HUD principal con estado de sensores y controles.

3.15.1 Inspector

Campo	Apunta a
joystickPanel	JoystickPanel
gpsStatusText	TextMeshProUGUI GPS
gyroStatusText	TextMeshProUGUI giroscopio

Campo	Apunta a
displacementText	TextMeshProUGUI posición
modeText	TextMeshProUGUI modo actual
toggleJoystickButton	Button toggle joystick
recalibrateButton	Button recalibrar
permissionGrantButton	Button permiso iOS
calibrationPanelButton	Button abrir calibración
cameraController	Main Camera

Capítulo 4

SATCS — Guía: Crear una Escena Nueva

Esta guía lleva paso a paso desde una escena vacía hasta una escena funcional con todos los sistemas activos.

Consejo: En lugar de crear desde cero, duplica la escena CASAS_OLAYA y modifica su contenido. Es más rápido y evita errores de configuración.

4.1 Paso 1 — Crear la escena

1. En la ventana **Project**, ir a Assets/Scenes/
 2. Clic derecho → **Create > Scene**
 3. Nombrarla según el barrio (ej: LA_CANTERA)
 4. Doble clic para abrirla
-

4.2 Paso 2 — GameObjects raíz (Managers)

Crea los siguientes GameObjects vacíos en la raíz de la escena (sin ser hijos de nada):

4.2.1 2.1 ARSceneSetup

- **GameObject > Create Empty** → nombre: ARSceneSetup
- Adjuntar script: ARSceneSetup

4.2.2 2.2 StageManager

- **GameObject > Create Empty** → nombre: StageManager
- Adjuntar script: StageManager
- En Inspector, el array stageConfigs tiene 6 entradas. Nombrarlas:

- [0] Intro · [1] Etapa1 · [2] Etapa2 · [3] Etapa3 · [4] Etapa4 · [5] Etapa5
- Los campos `objectsToActivate` y `objectsToDeactivate` se llenan más adelante

4.2.3 2.3 GPSManager

- **GameObject > Create Empty** → nombre: `GPSManager`
- Adjuntar script: `GPSManager`

4.2.4 2.4 GyroscopeManager

- **GameObject > Create Empty** → nombre: `GyroscopeManager`
- Adjuntar script: `GyroscopeManager`

4.2.5 2.5 CameraFeedManager

- **GameObject > Create Empty** → nombre: `CameraFeedManager`
- Adjuntar script: `CameraFeedManager`

4.2.6 2.6 AudioStageManager

- **GameObject > Create Empty** → nombre: `AudioStageManager`
- Adjuntar script: `AudioStageManager`
- Los `AudioSources` se crean automáticamente. Solo asignar clips en `stageAudios[0..5]`

4.2.7 2.7 CinematicManager

- **GameObject > Create Empty** → nombre: `CinematicManager`
- Adjuntar script: `CinematicManager`
- Adjuntar componente: **Video Player**
- En Inspector de `CinematicManager`: campo `videoPlayer` → arrastra el mismo `GameObject`

4.3 Paso 3 — Main Camera

1. Renombra la cámara principal a `Main Camera`
2. Verifica que tenga el **tag** `MainCamera`
3. Adjuntar script: `ARCameraController`
4. Adjuntar componente: **Character Controller**
5. Configurar el **Character Controller**:

Campo	Valor
Slope Limit	45
Step Offset	0.3
Skin Width	0.08
Radius	0.3
Height	1.7
Center Y	-0.85

6. Posicionar la cámara en $Y = 1.7$ sobre el suelo del terreno
7. En ARCameraController Inspector:
 - `forceJoystick = true` (para probar en editor sin GPS)

4.4 Paso 4 — Canvas Principal (AR_Canvas)

1. **GameObject > UI > Canvas** → nombre: AR_Canvas
2. En el componente **Canvas**:
 - Render Mode: Screen Space - Overlay
3. Adjuntar **Canvas Scaler**:
 - UI Scale Mode: Scale With Screen Size
 - Reference Resolution: 390 x 844
 - Match: 0.5
4. Verificar que haya un **EventSystem** en la escena (se crea automáticamente con el Canvas)

4.5 Paso 5 — Paneles UI dentro de AR_Canvas

Crea los siguientes hijos dentro de AR_Canvas. Todos empiezan **inactivos** excepto WelcomePanel.

4.5.1 5.1 JoystickPanel

- **UI > Image** → nombre: JoystickPanel
- Adjuntar script: JoystickController
- Crear hijo **Image** → nombre: Knob
- En JoystickController: knob → arrastra Knob
- Posicionar en esquina inferior izquierda
- **Desactivar** (Inspector → desmarcar el checkbox del nombre)

4.5.2 5.2 HotspotPanel

- **UI > Panel** → nombre: HotspotPanel

- Adjuntar script: HotspotUIPanel
- Crear hijos: TitleText (TMP), DescriptionText (TMP), Icon (Image), CloseButton (Button)
- Asignar referencias en el Inspector de HotspotUIPanel
- **Desactivar**

4.5.3 5.3 WelcomePanel

- **UI > Panel** → nombre: WelcomePanel
- Adjuntar script: WelcomePanel
- Crear hijos: slides como GameObjects, botones Anterior/Siguiente/Comenzar/Saltar
- Asignar en Inspector: slides[], prevButton, nextButton, startButton, skipButton
- **Dejar activo** (es el onboarding inicial)

4.5.4 5.4 NpcDialoguePanel

- **UI > Panel** → nombre: NpcDialoguePanel
- Adjuntar script: NpcDialoguePanel
- Crear hijos:
 - NpcPhoto (Image)
 - NpcName (TextMeshProUGUI)
 - DialogueText (TextMeshProUGUI)
 - MultipleChoicePanelGO → adjuntar script MultipleChoicePanel
 - * Dentro: OptionsContainer (Vertical Layout), FeedbackSection con FeedbackText y RetryButton
- Asignar en Inspector de NpcDialoguePanel: npcPhoto, npcNameText, dialogueText, choicePanel
- **Desactivar**

4.5.5 5.5 SiataCallPanel

- **UI > Panel** → nombre: SiataCallPanel
- Adjuntar script: SiataCallPanel
- Crear hijos:
 - CallerPhoto (Image), CallerName (TMP), CallStatus (TMP)
 - DialogueText (TMP)
 - MultipleChoicePanelGO → script MultipleChoicePanel
 - HangUpButton (Button)
- Asignar en Inspector: todos los campos
- **Desactivar**

4.5.6 5.6 SensorCalibPanel

- **UI > Panel** → nombre: SensorCalibPanel
- Adjuntar script: SensorCalibrationPanel
- Crear hijos: PitchSlider, RollSlider, YawSlider, ResetButton, RecalibrateButton, CloseButton

- **Desactivar**

4.5.7 5.7 CinematicPanel

- **UI > Panel** → nombre: CinematicPanel
- Crear hijos:
 - VideoRawImage (RawImage) — stretch fullscreen
 - LoadingText (TMP) — “Cargando video...”
 - SkipButton (Button) — esquina inferior derecha
- Asignar en Inspector de CinematicManager (que está en la raíz): cinematicPanel, videoDisplay, loadingText, skipButton
- **Desactivar**

4.5.8 5.8 CriticalModePanel

- **UI > Panel** → nombre: CriticalModePanel
- Adjuntar script: CriticalModePanel
- Crear hijos:
 - Overlay (Image negro semitransparente, fullscreen)
 - ContentBox (Image panel centrado) con:
 - * AlertIcon (Image)
 - * TitleText (TMP)
 - * DescriptionText (TMP)
 - * ContinueButton (Button “Entendido”)
- Asignar en Inspector: alertIcon, titleText, descriptionText, continueButton
- **Desactivar**

4.5.9 5.9 HUD

- **UI > Panel** (sin imagen de fondo) → nombre: HUD
- Adjuntar script: UIManager
- Crear textos e botones según necesidad
- Asignar todos los campos en Inspector de UIManager

4.6 Paso 6 — Conexiones de Inspector globales

Una vez creados todos los objetos, conectar:

Componente	Campo	Conectar con
StageManager	cameraController	Main Camera
ARCameraController	joystickController	JoystickPanel
UIManager	cameraController	Main Camera
UIManager	joystickPanel	JoystickPanel

Componente	Campo	Conectar con
NpcDialoguePanel	choicePanel	MultipleChoicePanelGO (hijo de NpcDialoguePanel)
SiataCallPanel	choicePanel	MultipleChoicePanelGO (hijo de SiataCallPanel)
SiataCallPanel	hangUpButton	HangUpButton

4.7 Paso 7 — Terreno

El **Character Controller** necesita colisiones para que el jugador no caiga.

- Si usas el componente **Terrain** de Unity → ya tiene collider integrado. No hacer nada.
- Si usas meshes importados (FBX) → seleccionarlos en Project → en Import Settings activar **Generate Mesh Colliders**, o agregar **Mesh Collider** manualmente en la escena.

4.8 Paso 8 — Agregar hotspots

Ver la guía completa en **05_FLUJO_TRABAJO.md — Sección: Agregar un hotspot**.

4.9 Paso 9 — Probar en editor

1. Activar `forceJoystick = true` en `ARCameraController` (GPS no funciona en editor)
2. Presionar **Play**
3. Usar **WASD** para moverse, **botón derecho del mouse** para rotar la cámara
4. Verificar que el `WelcomePanel` aparece y el botón “Comenzar” funciona
5. Acercarse a los hotspots para verificar que se activan

4.10 Paso 10 — Build WebGL

1. **File > Build Settings**
2. Platform: **WebGL** → Switch Platform
3. Verificar que la escena esté en la lista
4. **Player Settings:**
 - Company/Product Name: SATCS

- WebGL Template: Default o personalizado
5. **Build And Run**
 6. Para GPS y giroscopio: el servidor debe servir bajo **HTTPS**

Capítulo 5

SATCS — Flujo de Trabajo

Guías prácticas para las tareas más comunes del proyecto.

5.1 Agregar un Hotspot nuevo

Un hotspot es un punto de interés en la escena 3D que el jugador activa por proximidad o clic.

5.1.1 Paso 1 — Crear el ScriptableObject HotspotData

1. En **Project**, ir a la carpeta Assets/Hotspot/ (o donde estén los hotspots existentes)
2. Clic derecho → **Create > AR > Hotspot Data**
3. Nombrar el asset (ej: HS_VecinoJuan_Etapa1)
4. Configurar en Inspector:

Campo	Ejemplo
Title	"Vecino Juan"
Description	"¿Sabes qué hacer cuando suena la alarma?"
Action Type	NpcConversation
Required Stage	1 (solo visible en Etapa1)
Trigger Radius	5
Is Blinking	☐

5.1.2 Paso 2 — Crear el diálogo (si es NpcConversation o SiataCall)

1. Clic derecho → **Create > AR > NPC Dialogue Data**
2. Nombrar (ej: Dialogue_VecinoJuan)

3. Configurar:

NPC Name: "Juan, vecino del barrio"

NPC Text: "¿Cuál es el primer paso al escuchar la alarma del SIATA?"

Advances Stage On Correct: ✓

Correct Answer Delay: 1.5

Options[0]: "Llamar a la familia" – isCorrect: x – Feedback: "No olvides..."

Options[1]: "Evacuar hacia la ruta segura" – isCorrect: ✓ –

Feedback: "¡Correcto! La ruta..."

Options[2]: "Quedarse en casa" – isCorrect: x – Feedback: "Esto puede ser peligroso..."

4. En el HotspotData, asignar este asset en el campo Dialogue Data

5.1.3 Paso 3 — Colocar el objeto 3D en la escena

1. Arrastrar un modelo 3D (o usar un cubo como placeholder) a la escena
2. Posicionarlo donde deba estar el hotspot
3. Asegurarse de que tenga un **Collider** (Box, Sphere o Mesh Collider)
4. Adjuntar script: **HotspotController**
5. En Inspector de HotspotController:
 - data → asignar el HotspotData creado en el Paso 1
 - uiPanel → se auto-detecta, o asignar el HotspotPanel del Canvas

5.1.4 Paso 4 — Asignar al StageManager (opcional)

Si el hotspot debe aparecer/desaparecer al cambiar de etapa, agrégalo a stageConfigs: - En StageManager, campo stageConfigs[1] (Etapa1) - objectsToActivate → arrastrar el GameObject del hotspot

Alternativa: usar requiredStage = 1 en HotspotData. El HotspotController se activa/desactiva automáticamente según la etapa.

5.2 Agregar una cinemática

5.2.1 Paso 1 — Preparar el video

- Formato recomendado: .mp4 (H.264) para WebGL
- Para preview en editor Linux: convertir a .webm
- Copiar el archivo a Assets/StreamingAssets/Videos/

5.2.2 Paso 2 — Crear el HotspotData

1. **Create > AR > Hotspot Data**
2. Configurar:
 - Action Type = Cinematic
 - Cinematic Url = StreamingAssets/Videos/MiVideo.mp4

- Cinematic Clip = (opcional) el VideoClip importado para editor
- Cinematic Advances Stage = ☐ (si debe avanzar etapa al terminar)

5.2.3 Paso 3 — Verificar CinematicManager en escena

- Debe existir un GameObject CinematicManager con los scripts y el VideoPlayer
 - El CinematicPanel debe estar en el Canvas con sus referencias asignadas
 - Ver 04_CREAR_ESCENA.md paso 5.7
-

5.3 Crear un diálogo con el SIATA

Similar a NpcConversation pero con apariencia de llamada telefónica.

1. Crear NpcDialogueData (igual que antes)
 2. En HotspotData, asignar Action Type = SiataCall
 3. Asignar el NpcDialogueData en Dialogue Data
 4. El SiataCallPanel mostrará automáticamente “Llamando...” durante connectingDelay segundos antes de mostrar las opciones
-

5.4 Configurar el audio por etapa

1. Seleccionar el GameObject AudioStageManager en la escena
2. En Inspector, expandir stageAudios:
 - [0] Intro: asignar clip de ambiente tranquilo, volume 0.4
 - [1] Etapa1: clip ambiente barrio normal, volume 0.5
 - [2] Etapa2: clip lluvia leve, volume 0.5
 - [3] Etapa3: clip alarma + lluvia, volume 0.7, fadeTime 0.5
 - [4] Etapa4: clip alarma máxima, volume 0.8, fadeTime 0.3
 - [5] Etapa5: clip post-evento, volume 0.5

Para reproducir un sonido puntual (ej: trueno) desde cualquier script:

```
AudioStageManager.Instance.PlaySFX(miClip);
```

5.5 Configurar qué aparece en cada etapa

El StageManager puede activar/desactivar GameObjects automáticamente al cambiar de etapa.

1. Seleccionar StageManager en la escena
2. Expandir stageConfigs
3. Para cada etapa:
 - objectsToActivate[] → objetos que aparecen al entrar a esta etapa

- `objectsToDeactivate[]` → objetos que desaparecen

Ejemplo para Etapa3 (lluvia fuerte):

```
stageConfigs[3] – Etapa3
  objectsToActivate:  [LluviaFuerte, NivelAgua_Alto, AlarmaVisual]
  objectsToDeactivate: [LluviaLeve, NivelAgua_Normal]
```

5.6 Configurar el CriticalModePanel

El modal de modo crítico ya viene preconfigurado para Etapa3 y Etapa4. Para modificarlo:

1. Seleccionar `CriticalModePanel` en la jerarquía
 2. En Inspector, expandir `stageContents`:
 - `[0] stage: Etapa3, title, description, icon`
 - `[1] stage: Etapa4, title, description, icon`
 3. Para agregar una etapa nueva, aumentar el tamaño del array
-

5.7 Probar sin GPS (modo editor)

El GPS no funciona en el editor de Unity. Para probar el movimiento:

1. Seleccionar `Main Camera`
 2. En `ARCameraController`, activar `Force Joystick = true`
 3. Presionar **Play**
 4. Controles:
 - **WASD / flechas**: mover
 - **Botón derecho del mouse** + arrastrar: rotar cámara
 - (En móvil se usa el joystick táctil y el giroscopio)
-

5.8 Forzar una etapa específica al iniciar

Útil para probar una etapa sin pasar por las anteriores:

1. Seleccionar `StageManager`
2. Cambiar `startStage` a la etapa deseada (ej: Etapa3)
3. Presionar **Play**

Recuerda volver a Intro antes de hacer el build final.

5.9 Ajustar el giroscopio (en dispositivo real)

Si la cámara no apunta donde debería en el dispositivo:

1. En la escena, buscar el panel `SensorCalibPanel`
 2. En `UIManager`, verificar que `calibrationPanelButton` apunte a un botón del HUD
 3. En el dispositivo, presionar ese botón → aparecen 3 sliders (Pitch, Roll, Yaw)
 4. Ajustar hasta que la cámara apunte correctamente
 5. Los valores se guardan automáticamente
-

5.10 Recalibrar el GPS

Si el jugador “se teletransportó” o el GPS perdió la señal:

// Desde cualquier script:

```
ARCameraController cameraController = FindObjectOfType<ARCameraController>();  
cameraController.Recalibrate();
```

O presionar el botón `recalibrateButton` en el HUD (si está asignado en `UIManager`).

Capítulo 6

SATCS — Limitaciones y Consideraciones

6.1 Plataforma

6.1.1 WebGL es obligatorio

El proyecto está diseñado exclusivamente para WebGL. No funciona como aplicación nativa (APK, iOS, escritorio). Todo el código respeta esta restricción.

6.1.2 HTTPS requerido para GPS y giroscopio

Los navegadores modernos (Chrome, Safari, Firefox) solo permiten acceso a GPS y giroscopio desde páginas servidas bajo **HTTPS**. Bajo HTTP (incluso localhost) estos sensores no están disponibles.

- Para desarrollo local: usar un servidor con certificado autofirmado o localhost (Chrome a veces permite localhost sin HTTPS).
- Para producción: el servidor de hosting debe tener HTTPS activo.

6.1.3 iOS requiere permiso explícito de usuario

En iPhone/iPad, el acceso al giroscopio requiere que el usuario presione un botón que llame a `requestPermission()`. El botón `permissionGrantButton` en el HUD gestiona este flujo. Sin este paso, el giroscopio siempre devuelve datos vacíos en iOS.

6.2 GPS

6.2.1 No funciona en el editor de Unity

El plugin `ARSensors.jslib` solo funciona compilado en WebGL. En el editor, `GPSManager.IsAvailable` siempre devuelve `false`. Para probar en editor, activar `forceJoystick = true` en `ARCameraController` y usar WASD.

6.2.2 Primera lectura como origen

La primera coordenada GPS que llega se guarda como el punto de origen del jugador. Si el GPS tarda en obtener señal o llega con error, el origen puede quedar mal posicionado. Usar el botón **Recalibrar** para reiniciarlo.

6.2.3 Precisión del GPS en interiores

El GPS de teléfonos móviles tiene precisión de $\pm 3-15$ metros en exteriores. En interiores la señal se degrada notablemente. El juego está diseñado para usarse al aire libre.

6.2.4 Escala GPS $\leftrightarrow$ Unity

El campo `gpsToUnityScale` en `ARCameraController` convierte metros reales a unidades Unity. Si el jugador se mueve demasiado rápido o lento respecto al mundo 3D, ajustar este valor.

6.3 Video (CinematicManager)

6.3.1 WebGL: solo URL, no VideoClip

En builds WebGL, el `VideoPlayer` no puede usar `VideoClip` importados. Solo puede reproducir videos por URL (streaming). Los videos deben estar en `Assets/StreamingAssets/Videos/` y el campo `cinematicUrl` debe apuntar a ellos.

6.3.2 Editor Linux: mp4 puede fallar

El editor de Unity en Linux tiene soporte limitado para decodificación de mp4 en tiempo de edición. Si el video no se importa como `VideoClip`, convertirlo a `.webm` con:

```
ffmpeg -i video.mp4 -c:v libvpx-vp9 -crf 30 -b:v 0 video.webm
```

El build WebGL final sí reproduce mp4 correctamente (el navegador maneja el codec).

6.3.3 Sin soporte de DRM

Los videos se sirven directamente desde `StreamingAssets` sin cifrado. No usar videos con restricciones de derechos de autor.

6.4 CharacterController y Terreno

6.4.1 El terreno debe tener Collider

Sin un Collider en el terreno, el jugador cae indefinidamente por efecto de la gravedad. Verificar: - **Terrain** de Unity: ya incluye collider. No hace falta nada. - **Meshes importados (FBX)**: agregar Mesh Collider o activar Generate Mesh Colliders en Import Settings.

6.4.2 Configuración del CharacterController en Main Camera

Valores incorrectos pueden hacer que el jugador quede enterrado en el suelo o que flote:

Campo	Valor correcto	Problema si está mal
Height	1.7	Muy bajo: la cámara queda al ras del suelo. Muy alto: el jugador “se eleva”
Center Y	-0.85	Si no es -height/2, la cámara no queda a la altura de los ojos
Step Offset	0.3	Muy alto: sube paredes. Muy bajo: se traba en escalones pequeños

6.4.3 Sin soporte de salto

El sistema no implementa salto. La gravedad solo mantiene al jugador sobre el suelo. Esta es una decisión de diseño intencional para simplificar la navegación.

6.5 Audio

6.5.1 AudioStageManager necesita clips asignados

Si un `stageAudios[n].clip` queda en null, esa etapa tendrá silencio. Esto es válido y no genera errores.

6.5.2 Crossfade puede sonar raro con clips muy cortos

Si un clip de audio dura menos que el `fadeTime`, el crossfade puede quedar incompleto. Usar clips de al menos 5 segundos en loop.

6.6 Sistemas pendientes del GDD

Los siguientes sistemas están en el GDD pero aún no han sido implementados:

Sistema	Descripción
AerialViewController	Vista de cámara tipo dron en Etapa5. Actualmente la etapa existe pero sin cambio de perspectiva
RiskLevelIndicator	HUD con indicador N1-N4 de nivel de riesgo con colores. Actualmente no hay indicador visual de nivel
VisualEffectsStageController	Partículas de lluvia, cambio de skybox, iluminación dinámica por etapa. Actualmente los cambios visuales deben hacerse manualmente con <code>objectsToActivate/Deactivate</code> en <code>StageManager</code>
HotspotUIPanel enriquecido	El <code>InfoPanel</code> actual es básico (título + texto + ícono). El GDD define un panel con imagen de cabecera, scroll y nivel de riesgo

6.7 Limitaciones de diseño

6.7.1 Un solo diálogo correcto por hotspot

Cada `NpcDialogueData` tiene exactamente una opción `isCorrect = true`. El sistema no soporta diálogos con múltiples respuestas correctas.

6.7.2 El jugador no puede retroceder de etapa

`StageManager.NextStage()` solo avanza. No hay `PreviousStage()`. Si se necesita ir atrás, usar `GoToStage(Stage)` directamente desde código.

6.7.3 Hotspots en rango al inicio

Si un hotspot tiene `requiredStage = -1` (siempre visible) y el jugador aparece dentro de su radio, `DispatchAction()` se llama en el primer frame. Solución: posicionar el punto de inicio del jugador fuera de todos los radios de hotspot.

6.7.4 GPS y joystick no se combinan

El sistema es GPS o joystick. Si GPS está disponible y `forceJoystick = false`, el joystick táctil no tiene efecto sobre el movimiento. Sin embargo, WASD sí funciona siempre (se combina con joystick, no con GPS).

6.8 Preguntas frecuentes

¿Por qué el giroscopio no responde en iOS? Presionar el botón de permiso en el HUD (permissionGrantButton). iOS requiere que el usuario lo acepte explícitamente.

¿Por qué el jugador cae al vacío? El terreno no tiene Collider, o el jugador inicia por debajo del terreno. Agregar Mesh Collider al terreno y posicionar la Main Camera en $Y = 1.7$ sobre el suelo.

¿Por qué el video no reproduce en WebGL? Verificar que el archivo esté en Assets/StreamingAssets/Videos/ y que cinematicUrl en HotspotData tenga el path correcto (ej: StreamingAssets/Videos/clip.mp4). El servidor de WebGL debe servir el archivo con el MIME type correcto (video/mp4).

¿Por qué los hotspots no aparecen? Verificar el campo requiredStage en HotspotData. Si es 1 y el juego está en Etapa2, el hotspot estará inactivo. Usar -1 para hotspots siempre visibles.

¿Por qué el GPS posiciona mal al jugador? La primera coordenada que llega se usa como origen. Si el jugador estaba moviéndose o el GPS aún no había fijado señal, el origen puede ser incorrecto. Presionar **Recalibrar** para reiniciarlo.

Capítulo 7

SATCS — Documentación Técnica

Documentación del proyecto **Sistema de Alerta Temprana Comunitaria (SATCS)**.

7.1 Índice

Archivo	Contenido
01_OVERVIEW.md	Qué es el proyecto, tecnología, etapas, para quién es esta doc
02_ARQUITECTURA.md	Diagrama de sistemas, patrones de diseño, flujo de datos
03_SCRIPTS.md	Referencia de cada script: campos Inspector, métodos, notas
04_CREAR_ESCENA.md	Guía paso a paso para construir una escena desde cero
05_FLUJO_TRABAJO.md	Recetas prácticas: agregar hotspot, diálogo, video, audio, etapa
06_LIMITACIONES.md	Restricciones de plataforma, bugs conocidos, sistemas pendientes, FAQ

7.2 Inicio rápido

Solo quiero agregar un hotspot: → Ver 05_FLUJO_TRABAJO.md — Agregar un Hotspot nuevo

Quiero crear una escena nueva: → Ver 04_CREAR_ESCENA.md

Quiero entender cómo funciona el sistema: → Ver 02_ARQUITECTURA.md

Tengo un problema técnico: → Ver 06_LIMITACIONES.md — Preguntas frecuentes

7.3 Contexto para IAs

Para dar contexto completo a una IA al inicio de conversación, leer: - `AI_CONTEXT_SUMMARY.md` (en la raíz del proyecto) — resumen técnico completo - `CLAUDE_CONTEXT.md` (en la raíz del proyecto) — restricciones y forma de responder